COMPSCI 130 S1 2022
Introduction to Software Fundamentals

Binary Search Trees – Implementation

Agenda & Reading

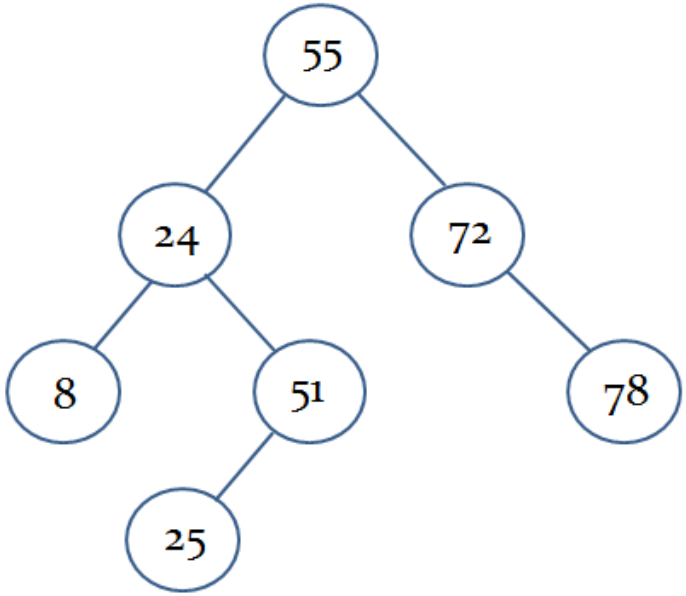
- Agenda
  - BinarySearchTree class
    - insert()
    - search()

- Online Coursebook:
  - https://onlinetextbook.auckland.ac.nz/11-binary-search-trees/

BinarySearchTree class

- The constructor is identical to the BinaryTree class constructor

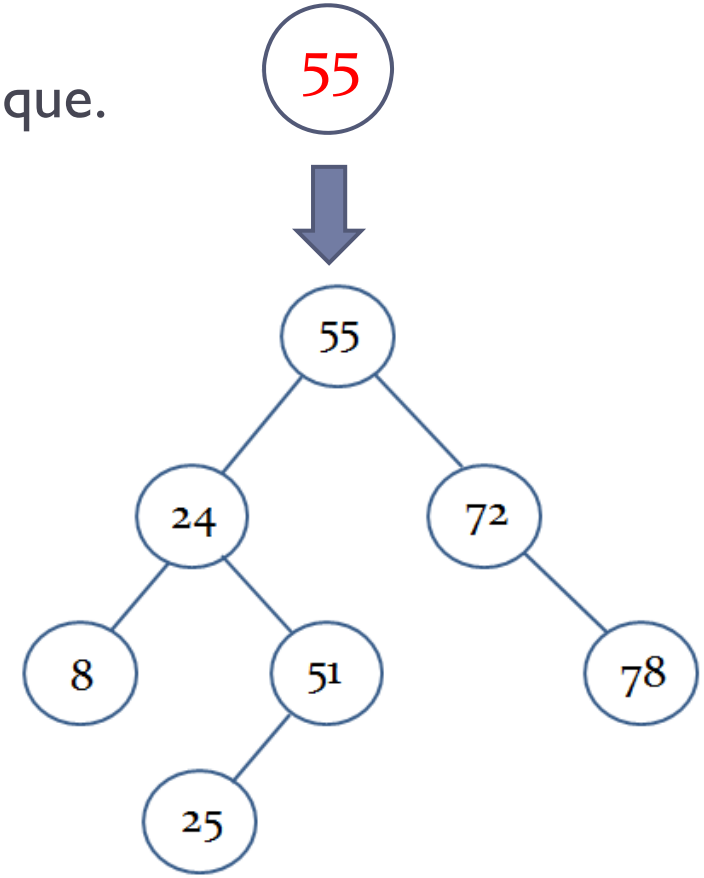
```
class BinarySearchTree:

    def __init__(self, data, left=None, right=None):
        self.data = data
        self.left = left
        self.right = right
```



Implementing the insert() method

- If you try to insert a value equal to the binary search tree data, do nothing.
  - Values in a binary search tree are unique.

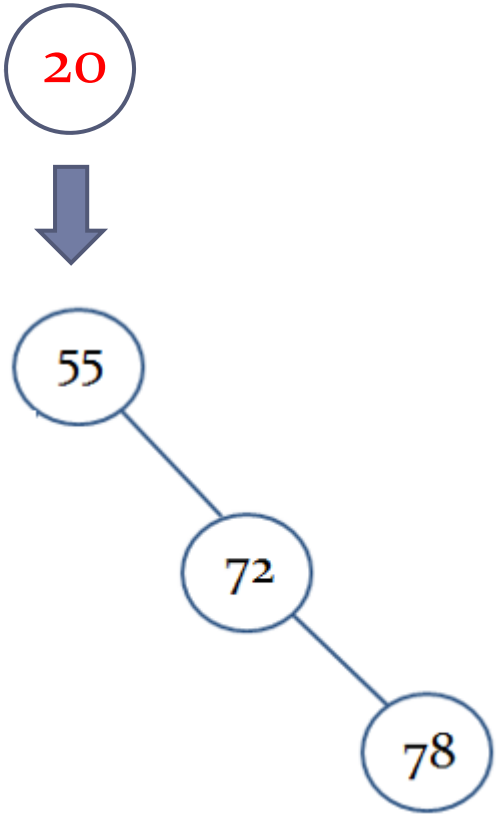
```
def insert(self, new_data):
    if new_data == self.data:
        return
```



Implementing the insert() method

- If you try to insert a value less than the binary search tree data, check the left subtree.

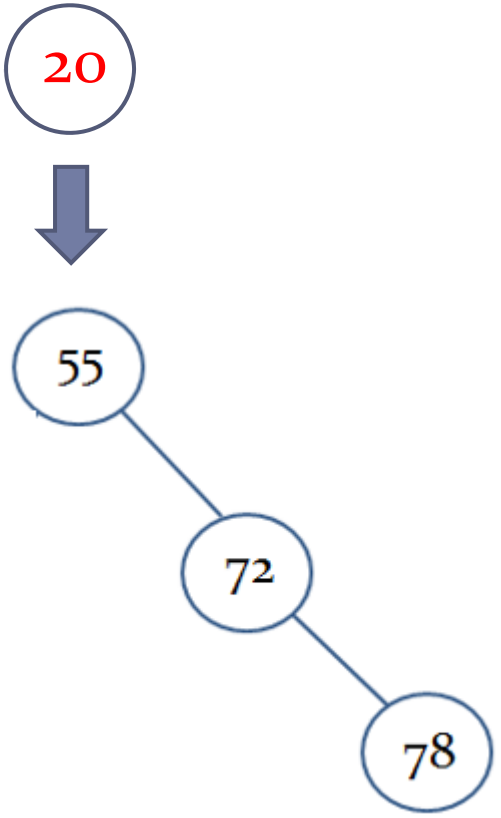
```
def insert(self, new_data):
    if new_data == self.data:
        return
    elif new_data < self.data:
```



Implementing the insert() method

- If you try to insert a value less than the binary search tree data, check the left subtree.
  - If there is no left subtree:

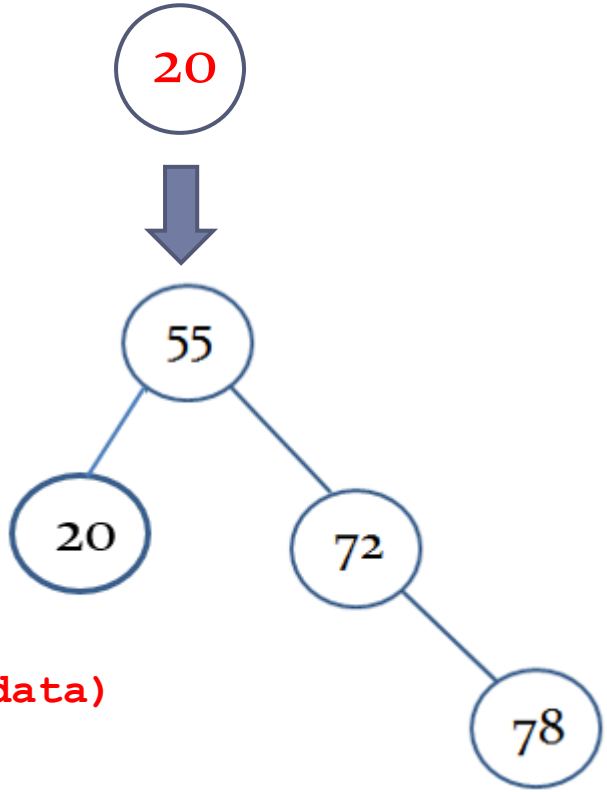
```
def insert(self, new_data):
    if new_data == self.data:
        return
    elif new_data < self.data:
        if self.left == None:
```



Implementing the insert() method

- If you try to insert a value less than the binary search tree data, check the left subtree.
  - If there is no left subtree:
    - create a new BinarySearchTree object containing this value and set it as the left subtree.

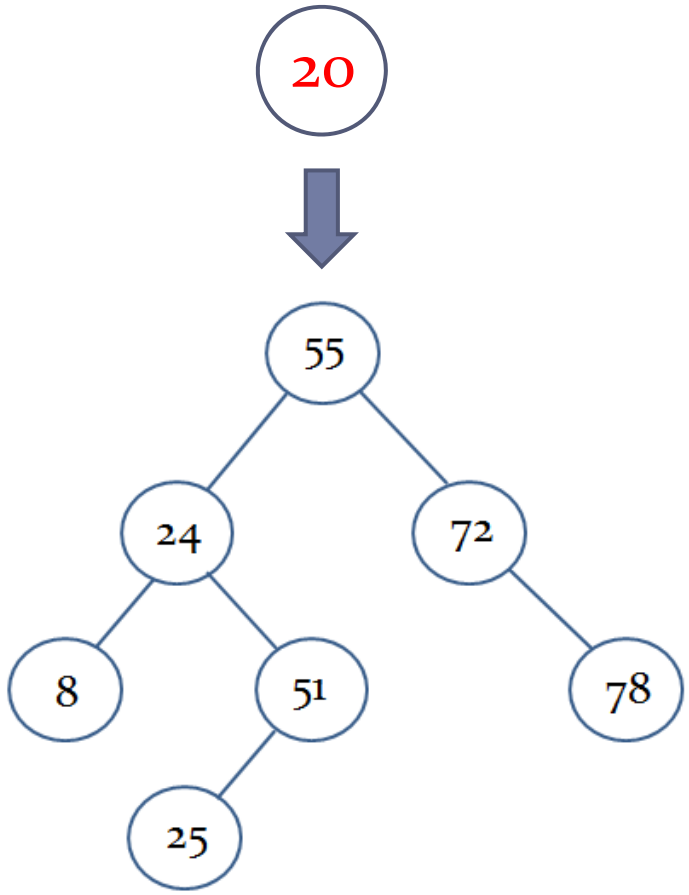
```
def insert(self, new_data):
    if new_data == self.data:
        return
    elif new_data < self.data:
        if self.left == None:
            self.left = BinarySearchTree(new_data)
```



Implementing the insert() method

- If you try to insert a value less than the binary search tree data, check the left subtree.
  - If there is no left subtree:
    - Create a new BinarySearchTree object containing this value and set it as the left subtree.
  - Else:
    - Insert the value into the left subtree.

```
def insert(self, new_data):
    if new_data == self.data:
        return
    elif new_data < self.data:
        if self.left == None:
            self.left = BinarySearchTree(new_data)
        else:
            self.left.insert(new_data)
```



Implementing the insert() method

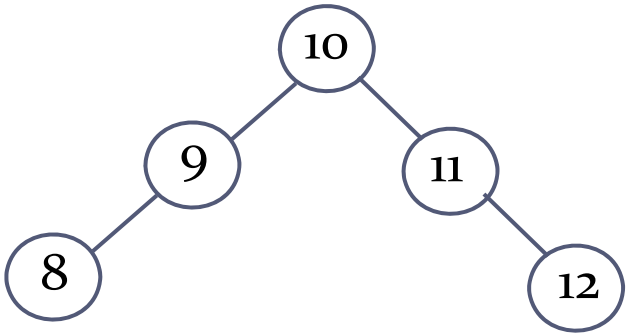
- A value greater than the binary search tree data, is inserted into the right subtree.

```
def insert(self, new_data):
    if new_data == self.data:
        return
    elif new_data < self.data:
        if self.left == None:
            self.left = BinarySearchTree(new_data)
        else:
            self.left.insert(new_data)
    else:
        if self.right == None:
            self.right = BinarySearchTree(new_data)
        else:
            self.right.insert(new_data)
```

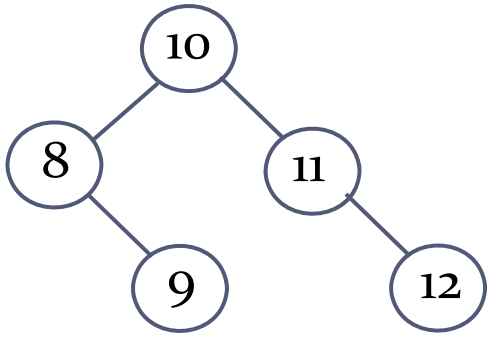
Order of insertion

- Consider the following BSTs:

```
search_tree = BinarySearchTree(10)

keys = [9, 11, 8, 12]
for value in keys:
    search_tree.insert(value)

print(search_tree)
```




```
search_tree = BinarySearchTree(10)

keys = [8, 9, 11, 12]
for value in keys:
    search_tree.insert(value)

print(search_tree)
```

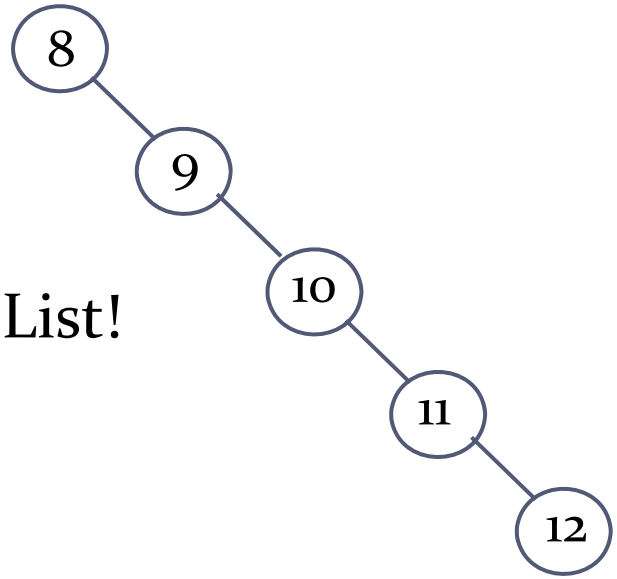


Order of insertion

- Consider the following BST:

```
search_tree = BinarySearchTree(8)

keys = [9, 10, 11, 12]
for value in keys:
    search_tree.insert(value)

print(search_tree)
```

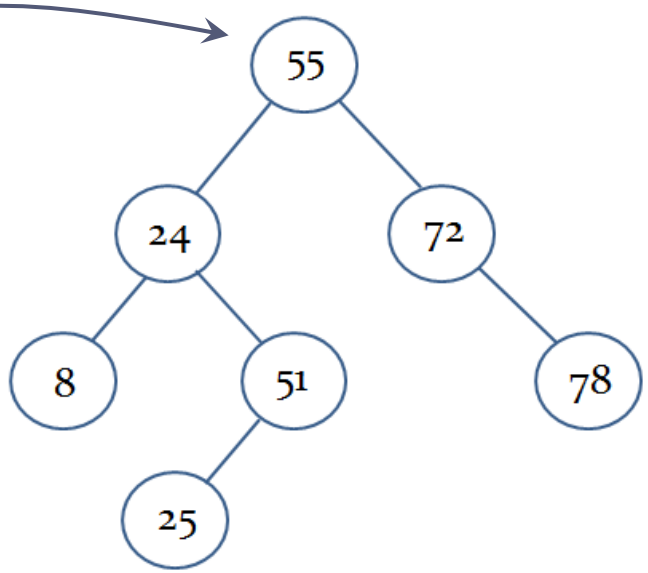
Degeneration to a Linked List!



Implementing the search() method

```
def search(self, find_data):
```

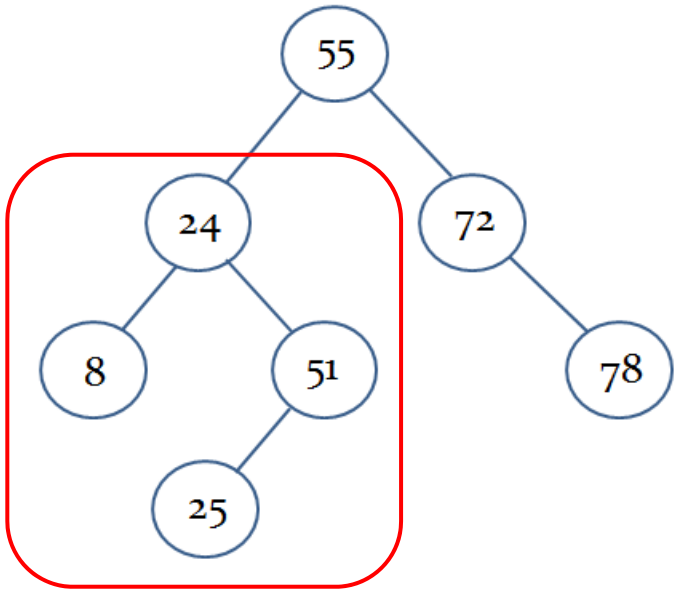
Check the root node



Implementing the search() method

```
def search(self, find_data):
```

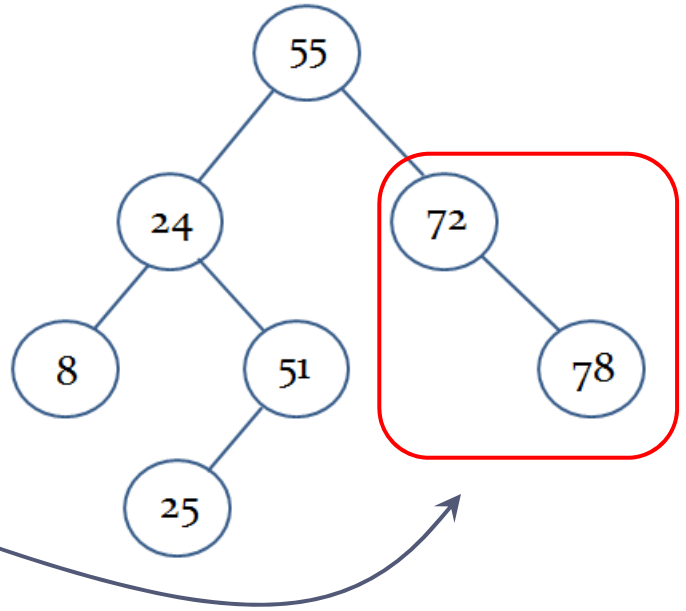
If the search value is less than the root node, then check the left subtree



Implementing the search() method

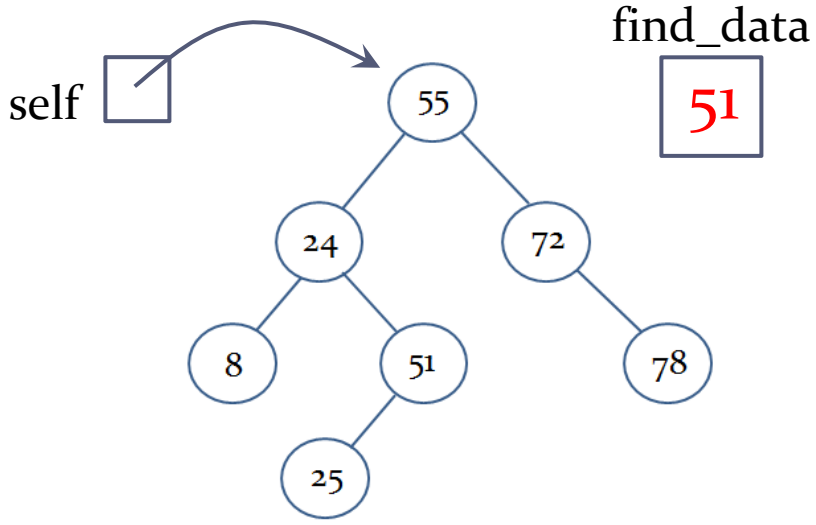
```
def search(self, find_data):
```

If the search value is greater than the root node, then check the right subtree



Implementing the search() method

```
def search(self, find_data):
    if self.data == find_data:
        return True
    elif find_data < self.data and self.left != None:
        return self.left.search(find_data)
    elif find_data > self.data and self.right != None:
        return self.right.search(find_data)
    else:
        return False
```



BinarySearchTree class so far

```
class BinarySearchTree:
    def __init__(self, data, left=None, right=None):
        self.data = data
        self.left = left
        self.right = right
    def insert(self, new_data):
        if new_data == self.data:
            return
        elif new_data < self.data:
            if self.left == None:
                self.left = BinarySearchTree(new_data)
            else:
                self.left.insert(new_data)
        else:
            if self.right == None:
                self.right = BinarySearchTree(new_data)
            else:
                self.right.insert(new_data)
    def search(self, find_data):
        if self.data == find_data:
            return True
        elif find_data < self.data and self.left != None:
            return self.left.search(find_data)
        elif find_data > self.data and self.right != None:
            return self.right.search(find_data)
        else:
            return False
```